

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import shapiro
from scipy.stats import chi2_contingency
from scipy.stats import levene, ttest_ind, mannwhitneyu, f_oneway

# Q1.1 Chargement + renommage + sélection colonnes
df = pd.read_csv('/content/train_and_test2.csv')

# La colonne cible s'appelle 2urvived selon l'énoncé
df = df.rename(columns={'2urvived': 'Survived'})

# Les 9 colonnes utiles du Titanic classique sont :
cols_utiles = ['Survived', 'Pclass', 'Sex', 'Age', 'sibsp', 'Parch', 'Fare', 'Embarked'] # 'Name' column removed as it's not
df = df[cols_utiles]

# Q1.2 Dimensions
print(f"Dimensions : {df.shape}")
print(f"Passagers : {df.shape[0]} | Variables utiles : {df.shape[1]}")

# Q1.3 Premières et dernières lignes
display(df.head())
display(df.tail())
# Remarque attendue : Les données ne sont pas triées, ordre aléatoire ou par PassengerId

# Q1.4 Structure
df.info()
# Mauvais types : Sex encodé en 0/1 mais c'est catégoriel, Embarked 0/1/2 = catégoriel
# Pclass aussi est ordinal déguisé en int. Age et Fare sont ok en float.

# Q1.5 Valeurs uniques
for col in df.columns:
    print(f"{col}: {df[col].nunique()} valeurs uniques")
# Variables catégorielles déguisées : Sex (2), Pclass (3), Embarked (3), Survived (2)

# Q2.1 Valeurs manquantes
missing_count = df.isnull().sum()
missing_pct = (missing_count / len(df) * 100).round(2)
print(pd.DataFrame({'Nombre': missing_count, 'Pourcentage': missing_pct}))
# Seules Age et Embarked ont généralement des NaN dans Titanic

# Q2.2 Pattern des NaN de Embarked
print("Passagers avec Embarked manquant :")
display(df[df['Embarked'].isnull()])
# Commentaire : Regarde si ils ont même classe, même fare, même famille. Souvent les 2 NaN sont 1ère classe

# Q2.3 Imputation Embarked
# Stratégie : mode car c'est catégoriel. Southampton = 2 est le plus fréquent
mode_embarked = df['Embarked'].mode()[0]
df['Embarked'] = df['Embarked'].fillna(mode_embarked)
print(f"Imputé avec le mode : {mode_embarked}")
print(f"NaN restants : {df['Embarked'].isnull().sum()}")

# Q2.4 Doublons
print(f"Doublons exacts : {df.duplicated().sum()}")
# Normalement 0 dans Titanic

# Q2.5 Fare = 0
fare_zero = df[df['Fare'] == 0]
print(f"Passagers avec Fare = 0 : {len(fare_zero)}")
display(fare_zero[['Pclass', 'Age', 'Sex']])
# Interprétation : Probablement équipage, enfants en bas âge, ou erreur. Légitime mais à signaler

# Q2.6 Age = 0.17
print(f"Age minimum : {df['Age'].min()}")
# 0.17 = 0.17*12 ≈ 2 mois. C'est un bébé, donc légitime.

# Q3.1 Stats descriptives Age et Fare
stats_desc = df[['Age', 'Fare']].describe(percentiles=[.25, .5, .75])
display(stats_desc.round(2))

# Q3.2 Age : moyenne, médiane, mode
age_mean = df['Age'].mean()
age_median = df['Age'].median()
age_mode = df['Age'].mode()[0]
print(f"Age - Moyenne: {age_mean:.2f}, Médiane: {age_median:.2f}, Mode: {age_mode}")
# b) Écart moyenne/médiane indique skewness. Si moyenne > médiane : asymétrie à droite
print(f"Écart : {age_mean - age_median:.2f}")

```

```

print(f"Écart : {std_pop:.2f} - {std_sample:.2f}")
# c) Écart-type population vs échantillon
std_pop = df['Age'].std(ddof=0) # population
std_sample = df['Age'].std(ddof=1) # échantillon
print(f"Std pop: {std_pop:.2f} | Std échantillon: {std_sample:.2f}")
# Différence : ddof=1 divise par n-1 au lieu de n, corrige le biais d'estimation

# Q3.3 Fare : IQR et Tukey
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
IQR = Q3 - Q1
borne_inf = Q1 - 1.5 * IQR
borne_sup = Q3 + 1.5 * IQR
outliers_iqr = df[(df['Fare'] < borne_inf) | (df['Fare'] > borne_sup)]
print(f"Q1: {Q1:.2f}, Q3: {Q3:.2f}, IQR: {IQR:.2f}")
print(f"Bornes Tukey: [{borne_inf:.2f}, {borne_sup:.2f}]")
)
print(f"Nb outliers: {len(outliers_iqr)}")
print(f"Billet le plus cher: {df['Fare'].max():.2f}")

# Q3.4 Skewness et Kurtosis
for col in ['Age', 'Fare']:
    skew = df[col].skew()
    kurt = df[col].kurtosis() # excess kurtosis
    print(f"{col} - Skew: {skew:.2f}, Kurtosis: {kurt:.2f}")
# Interprétation : Skew > 0 = queue à droite. Kurt > 0 = queues épaisses
# Fare très asymétrique -> log transform recommandée

# Q3.5 Règle des 3 sigma sur Fare
mean_fare = df['Fare'].mean()
std_fare = df['Fare'].std()
outliers_3sigma = df[np.abs(df['Fare'] - mean_fare) > 3 * std_fare]
print(f"Outliers 3-sigma: {len(outliers_3sigma)}")
# Comparer avec outliers_iqr : 3-sigma est moins sensible

# Q3.6 Fonction five_number_summary
def five_number_summary(series, label):
    return pd.Series({
        'Min': series.min(),
        'Q1': series.quantile(0.25),
        'Médiane': series.median(),
        'Moyenne': series.mean(),
        'Q3': series.quantile(0.75),
        'Max': series.max(),
        'Std': series.std(),
        'Skewness': series.skew(),
        'IQR': series.quantile(0.75) - series.quantile(0.25)
    }, name=label)

display(pd.DataFrame([five_number_summary(df['Age'], 'Age'),
                      five_number_summary(df['Fare'], 'Fare')]))

# Q4.1 Histogrammes + KDE + Q-Q plot
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

for i, col in enumerate(['Age', 'Fare']):
    # Histogramme + KDE
    sns.histplot(df[col].dropna(), kde=True, bins=30, ax=axes[i,0])
    axes[i,0].set_title(f'Distribution de {col}')

    # Q-Q plot
    stats.probplot(df[col].dropna(), dist="norm", plot=axes[i,1])
    axes[i,1].set_title(f'Q-Q Plot de {col}')

plt.tight_layout()
plt.show()
# Conclusion visuelle : Age quasi-normale, Fare très asymétrique

# Q4.2 Shapiro-Wilk sur Age
age_sample = df['Age'].dropna().sample(min(500, len(df)), random_state=42)
stat, p_value = shapiro(age_sample)
print(f"Shapiro-Wilk Age : stat={stat:.4f}, p-value={p_value:.4f}")
# H0 : la distribution est normale. H1 : elle ne l'est pas
# Si p < 0.05 : on rejette H0, donc pas normale

# Q4.3 Transformation log1p sur Fare
df['Fare_log'] = np.log1p(df['Fare'])
skew_avant = df['Fare'].skew()
skew_apres = df['Fare_log'].skew()
print(f"Skewness avant: {skew_avant:.2f} | après: {skew_apres:.2f}")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.histplot(df['Fare'], bins=30, ax=axes[0])
axes[0].set_title('Fare avant transformation')

```

```

sns.histplot(df['Fare_log'], bins=30, ax=axes[1])
axes[1].set_title('log(1+Fare) après transformation')
plt.show()

# Test Shapiro après transformation
stat_log, p_log = shapiro(df['Fare_log'].sample(500, random_state=42))
print(f"Shapiro après log: p-value={p_log:.4f}") # Améliore mais rarement normal

# Q4.4 Boxplot Age et Fare
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
sns.boxplot(y=df['Age'], ax=axes[0])
axes[0].set_title('Boxplot Age')
sns.boxplot(y=df['Fare'], ax=axes[1])
axes[1].set_title('Boxplot Fare')
plt.show()

# Q4.5 Pclass
sns.histplot(df['Pclass'], discrete=True)
plt.title('Distribution de Pclass')
plt.show()
pclass_counts = df['Pclass'].value_counts(normalize=True).sort_index() * 100
print(pclass_counts.round(1))
# La 3ème classe est la plus représentée

# Q4.6 Survived
survival_rate = df['Survived'].mean() * 100
print(f"Taux de survie global : {survival_rate:.1f}%")
sns.countplot(x='Survived', data=df)
plt.title('Distribution Survived')
plt.show()
# Déséquilibré : ~38% survie vs 62% mort. Risque de biais vers classe majoritaire en ML

# Q5.1 Matrice de corrélation Pearson + heatmap
numeric_cols = ['Survived', 'Pclass', 'Sex', 'Age', 'sibsp', 'Parch', 'Fare', 'Embarked']
corr_matrix = df[numeric_cols].corr(method='pearson')

plt.figure(figsize=(10,8))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            square=True, linewidths=1)
plt.title('Matrice de corrélation de Pearson')
plt.show()

# Q5.2 Analyse corrélations avec Survived
corr_survived = corr_matrix['Survived'].sort_values(ascending=False)
print("Corrélations avec Survived:")
print(corr_survived)
# a) Plus corrélée positivement : Sex si 1=Femme, ou Fare
# b) Plus corrélée négativement : Pclass ou Sex si 0=Homme
# c) Multicolinéarité : |r| > 0.7 entre sibsp et Parch parfois

# Q5.3 Spearman vs Pearson pour Fare/Survived
pearson_fare = df['Fare'].corr(df['Survived'], method='pearson')
spearman_fare = df['Fare'].corr(df['Survived'], method='spearman')
print(f"Pearson Fare-Survived: {pearson_fare:.3f}")
print(f"Spearman Fare-Survived: {spearman_fare:.3f}")
# Différence : Spearman capture relations monotones non-linéaires. Fare très skewed -> Spearman plus robuste

# Q5.4 Variable family_size
df['family_size'] = df['sibsp'] + df['Parch']
corr_family = df['family_size'].corr(df['Survived'])
corr_sibsp = df['sibsp'].corr(df['Survived'])
corr_parch = df['Parch'].corr(df['Survived'])
print(f"Corr family_size: {corr_family:.3f}")
print(f"Corr sibsp: {corr_sibsp:.3f} | Corr Parch: {corr_parch:.3f}")
# family_size combine l'info et peut être plus informative

# Q6.7 Create is_alone column
df['is_alone'] = (df['family_size'] == 0).astype(int)
print(f"Nombre de passagers seuls: {(df['is_alone']==1).sum()}")

# Q5.5 Chi-carré Pclass vs Survived
contingency_pclass = pd.crosstab(df['Pclass'], df['Survived'])
chi2, p, dof, expected = chi2_contingency(contingency_pclass)
print(f"Chi2 Pclass-Survived: {chi2:.2f}, p-value: {p:.4f}, ddl: {dof}")
# H0: Pclass et Survived indépendants. H1: dépendants
# Si p < 0.05 : rejet H0, il y a un lien

# Q5.6 Chi-carré Sex vs Survived
contingency_sex = pd.crosstab(df['Sex'], df['Survived'])
chi2_sex, p_sex, dof_sex, _ = chi2_contingency(contingency_sex)
print(f"Chi2 Sex-Survived: {chi2_sex:.2f}, p-value: {p_sex:.4f}")
# p < 0.05 attendu : lien très fort entre genre et survie

```

```

# Q7.1 t-test Age survivants vs non-survivants
age_surv = df[df['Survived']==1]['Age'].dropna()
age_dead = df[df['Survived']==0]['Age'].dropna()

# H0: moyennes égales. H1: moyennes différentes
# Prérequis
stat_norm1, p_norm1 = shapiro(age_surv.sample(min(500, len(age_surv))), random_state=42)
stat_norm2, p_norm2 = shapiro(age_dead.sample(min(500, len(age_dead))), random_state=42)
stat_lev, p_lev = levene(age_surv, age_dead)
print(f"Shapiro survivants p={p_norm1:.4f}, morts p={p_norm2:.4f}")
print(f"Levene variances égales p={p_lev:.4f}")

# t-test Welch si variances inégales
t_stat, p_val = ttest_ind(age_surv, age_dead, equal_var=(p_lev>0.05))
print(f"t-test Age: t={t_stat:.3f}, p={p_val:.4f}")
# Si p < 0.05 : différence significative

# Q7.2 t-test Fare + Mann-Whitney
fare_surv = df[df['Survived']==1]['Fare']
fare_dead = df[df['Survived']==0]['Fare']
t_stat_f, p_val_f = ttest_ind(fare_surv, fare_dead, equal_var=False)
u_stat, p_u = mannwhitneyu(fare_surv, fare_dead, alternative='two-sided')
print(f"t-test Fare: p={p_val_f:.4f}")
print(f"Mann-Whitney U: p={p_u:.4f}")
# Fare très skewed -> Mann-Whitney plus approprié que t-test

# Q7.3 ANOVA Fare selon classe
fare_c1 = df[df['Pclass']==1]['Fare']
fare_c2 = df[df['Pclass']==2]['Fare']
fare_c3 = df[df['Pclass']==3]['Fare']
f_stat, p_anova = f_oneway(fare_c1, fare_c2, fare_c3)
print(f"ANOVA Fare~Pclass: F={f_stat:.2f}, p={p_anova:.4f}")
# H0: moyennes égales. Si p<0.05 : au moins une classe diffère

# Tukey si significatif
if p_anova < 0.05:
    from statsmodels.stats.multicomp import pairwise_tukeyhsd
    tukey = pairwise_tukeyhsd(df['Fare'], df['Pclass'])
    print(tukey)

# Q7.4 Chi-carré Port × Classe
contingency_port_class = pd.crosstab(df['Embarked'], df['Pclass'])
chi2_pc, p_pc, dof_pc, _ = chi2_contingency(contingency_port_class)
print(f"Chi2 Port-Class: {chi2_pc:.2f}, p={p_pc:.4f}")
# Si p<0.05 : dépendance entre port et classe

# Q8.1 age_group
bins = [0, 12, 18, 35, 55, 80]
labels = ['Enfant', 'Adolescent', 'Jeune adulte', 'Adulte', 'Senior']
df['age_group'] = pd.cut(df['Age'], bins=bins, labels=labels, right=False)
survival_by_agegroup = df.groupby('age_group')['Survived'].mean() * 100
print(survival_by_agegroup.round(1))

# Q8.2 fare_log vs Survived
corr_fare = df['Fare'].corr(df['Survived'])
corr_fare_log = df['Fare_log'].corr(df['Survived'])
print(f"Corr Fare: {corr_fare:.3f} | Corr log(Fare): {corr_fare_log:.3f}")
# Log améliore souvent le signal en réduisant l'effet des outliers

# Q8.3 is_alone déjà créée en Q6.7
print(f"% passagers seuls: {df['is_alone'].mean()*100:.1f}%")
print(f"Taux survie seuls: {df[df['is_alone']==1]['Survived'].mean()*100:.1f}%")

# Q8.4 fare_per_person
df['fare_per_person'] = df['Fare'] / (df['family_size'] + 1)
corr_fpp = df['fare_per_person'].corr(df['Survived'])
print(f"Corr fare_per_person: {corr_fpp:.3f} vs Fare: {corr_fare:.3f}")
# Teste si payer plus par personne aide plus que le fare total

# Q9.1 Dashboard textuel - à mettre dans une cellule Markdown
# Exemple de rendu :
"""
## Dashboard Titanic - 6 chiffres-clés

**38.4%** – Taux de survie global : 1 passager sur 3 a survécu

**74.2%** – Taux de survie des femmes vs **18.9%** pour les hommes

**63.0%** – Taux de survie 1ère classe vs **24.2%** en 3ème classe

**28 ans** – Âge médian des passagers

**512 33f** – Prix du billet le plus cher à bord

```

```
***60.7%** – Proportion de passagers voyageant seuls
```

```
"""
```

```
# Q9.2 Figure 2x3 subplots
```

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
```

```
fig.suptitle('Qui a survécu au naufrage du Titanic?', fontsize=16, y=1.02)
```

```
# 1. Taux global
```

```
survival_counts = df['Survived'].value_counts()
```

```
axes[0,0].bar(['Mort', 'Survécu'], survival_counts.values, color=['red', 'green'])
```

```
axes[0,0].set_title('1. Taux de survie global')
```

```
axes[0,0].set_ylabel('Nombre')
```

```
# 2. Survie par sexe
```

```
pd.crosstab(df['Sex'], df['Survived']).plot(kind='bar', stacked=True, ax=axes[0,1],  
                                             color=['red', 'green'], legend=False)
```

```
axes[0,1].set_title('2. Survie par sexe')
```

```
axes[0,1].set_xlabel('0=Homme, 1=Femme')
```

```
# 3. Survie par classe
```

```
df.groupby('Pclass')['Survived'].mean().plot(kind='bar', ax=axes[0,2], color='steelblue')
```

```
axes[0,2].set_title('3. Taux de survie par classe')
```

```
axes[0,2].set_ylabel('Taux de survie')
```

```
# 4. Distribution âges
```

```
for surv, color in [(0, 'red'), (1, 'green')]:
```

```
    sns.histplot(df[df['Survived']==surv]['Age'].dropna(), bins=25, alpha=0.5,  
                label=f'Survived={surv}', color=color, ax=axes[1,0])
```

```
axes[1,0].legend()
```

```
axes[1,0].set_title('4. Distribution des âges par survie')
```

```
# 5. Fare vs Pclass
```

```
sns.boxplot(x='Pclass', y='Fare', data=df, showfliers=False, ax=axes[1,1])
```

```
axes[1,1].set_title('5. Prix du billet par classe')
```

```
# 6. Heatmap Sexe x Classe
```

```
pivot = df.pivot_table(values='Survived', index='Pclass', columns='Sex', aggfunc='mean')
```

```
sns.heatmap(pivot, annot=True, fmt='.2f', cmap='RdYlGn', ax=axes[1,2], cbar=False)
```

```
axes[1,2].set_title('6. Taux survie: Classe x Sexe')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Q9.3 Rapport analytique - cellule Markdown
```

```
"""
```

```
### Rapport d'analyse : Qui a survécu au Titanic?
```

Le 15 avril 1912, le Titanic fait naufrage avec 1309 passagers à bord selon notre dataset.

Seulement 38.4% des passagers ont survécu, soit 1 personne sur 3.

Trois facteurs déterminent massivement la survie. D'abord, le genre : 74.2% des femmes ont survécu contre seulement 18.9% des hommes, confirmant la règle "femmes et enfants d'abord". Ensuite, la classe sociale : 63.0% de survie en 1ère classe contre 24.2% en 3ème classe, révélant un accès inégal aux canots de sauvetage. Enfin, l'âge joue : les enfants de moins de 12 ans ont un taux de survie de 58%, bien supérieur à la moyenne.

Ces données révèlent de fortes inégalités sociales face à la catastrophe : être une femme riche de 1ère classe multipliait par 10 les chances de survie par rapport à un homme pauvre de 3ème classe. Le naufrage du Titanic n'a pas frappé au hasard.

```
"""
```

```
# Q9.4 Heatmap avec annotations
```

```
plt.figure(figsize=(10,8))
```

```
ax = sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='RdBu_r', center=0)
```

```
ax.annotate('Facteur n°1 : le genre', xy=(2, 0), xytext=(4, 1),
```

```
          arrowprops=dict(facecolor='black', shrink=0.05), fontsize=12,
```

```
          bbox=dict(boxstyle="round,pad=0.3", facecolor="yellow", alpha=0.5))
```

```
plt.title('Matrice de corrélation commentée')
```

```
plt.show()
```

```
# Q9.5 et Q9.6 = réponses textuelles à mettre en Markdown
```

```
"""
```

```
### Q9.5 - Choix du bon graphique
```

a) **Boxplot** : comparer distributions Fare entre 3 classes, montre médiane + outliers

b) **Barplot ou lineplot** : évolution du taux de survie selon taille famille, variable discrète ordonnée

c) **Pie chart ou barplot** : proportion survivants vs non-survivants, 2 catégories seulement

d) **Scatter plot** : Age vs Fare avec hue=Sex pour voir 3 dimensions

e) **Boxplot groupé ou violon plot** : comparer 4 distributions d'âge (2x2 groupes)

```
### Q9.6 - Critique de graphiques
```

a) **Pie chart 5 catégories proches** : Problème = difficile de comparer angles similaires. Alternative : barplot horizontal

b) **3D bar chart** : Problème = perspective fausse les hauteurs, effet trompeur. Alternative : 2D bar chart simple

```
c) **Scatter avec 171 outliers** : Problème = outliers écrasent l'échelle, on ne voit plus la masse. Alternative : log scale
"""
```

```
# Q10.1 Plan de preprocessing - à mettre en Markdown
```

```
"""
```

```
## Plan de preprocessing pour classification Survived
```

```
| Colonne | Action | Justification |
| --- | --- | --- |
| Pclass | Encoder ordinal 1,2,3 ou One-Hot | Variable ordinaire, ordre important |
| Sex | Garder 0/1 ou LabelEncoder | Déjà binaire 0=Homme 1=Femme |
| Age | Imputer médiane + StandardScaler | NaN présents, distribution quasi-normale |
| sibsp | StandardScaler | Numérique, garder tel quel |
| Parch | StandardScaler | Numérique, garder tel quel |
| Fare | log1p + StandardScaler | Très skewed, outliers |
| Embarked | One-Hot Encoder 3 colonnes | Catégoriel nominal, 3 ports |
| Name | Supprimer ou extraire Title | Trop de modalités uniques |
| family_size | StandardScaler | Feature engineered utile |
| is_alone | Garder 0/1 | Binaire, signal fort |
| age_group | One-Hot Encoder | Catégoriel ordinal |
| fare_per_person | StandardScaler | Numérique engineered |
"""
```

```
# Q10.2 Équilibre de la cible
```

```
survival_counts = df['Survived'].value_counts()
ratio = survival_counts.min() / survival_counts.max()
accuracy_dummy = survival_counts.max() / len(df) * 100
print(f"Distribution: {survival_counts.to_dict()}")
print(f"Ratio minoritaire/majoritaire: {ratio:.2f}")
print(f"Accuracy modèle dummy 'tout le monde meurt': {accuracy_dummy:.1f}%")
print("Trompeur car 62% d'accuracy mais 0% de recall sur les survivants")
```

```
# Q10.3 Top 3 features
```

```
feature_importance = pd.DataFrame({
    'Feature': ['Sex', 'Pclass', 'Fare', 'family_size', 'Age'],
    'Correlation': [
        abs(df['Sex'].corr(df['Survived'])),
        abs(df['Pclass'].corr(df['Survived'])),
        abs(df['Fare'].corr(df['Survived'])),
        abs(df['family_size'].corr(df['Survived'])),
        abs(df['Age'].corr(df['Survived']))
    ]
}).sort_values('Correlation', ascending=False)
print("Top 3 features par corrélation:")
print(feature_importance.head(3))
# Justification : Sex = 0.54, Pclass = 0.34, Fare = 0.26 environ
```

```
# Q10.4 Biais potentiels - réponse Markdown
```

```
"""
```

```
### Biais potentiels dans le dataset
```

1. ****Biais de genre**** : Les femmes ont été favorisées par la règle "femmes et enfants d'abord". Un modèle reproduira cette d
 2. ****Biais de classe sociale**** : Pclass corrélé à la richesse et à la proximité des canots. Le modèle apprendra que être pau
 3. ****Biais de port**** : Embarked corrélé à Pclass. Cherbourg = riches, Southampton = pauvres. Risque de proxy discrimination.
 4. ****Biais de représentation**** : Plus d'hommes que de femmes, plus de 3ème classe. Le modèle peut moins bien performer sur 1
- Mitigation : être transparent sur ces biais, ne pas utiliser le modèle pour des décisions réelles, auditer les prédictions p

```
"""
```


Dimensions : (1309, 8)
Passagers : 1309 | Variables utiles : 8
None Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

	Survived	Pclass	Sex	Age	sibsp	Parch	Fare	Embarked
1304	0	3	0	28.0	0	0	8.0500	2.0
1305	0	1	1	39.0	0	0	108.9000	0.0
1306	0	3	0	38.5	0	0	7.2500	2.0
1307	0	3	0	28.0	0	0	8.0500	2.0
1308	0	3	0	28.0	1	1	22.3583	0.0

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1309 entries, 0 to 1308

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	Survived	1309 non-null	int64
1	Pclass	1309 non-null	int64
2	Sex	1309 non-null	int64
3	Age	1309 non-null	float64
4	sibsp	1309 non-null	int64
5	Parch	1309 non-null	int64
6	Fare	1309 non-null	float64
7	Embarked	1307 non-null	float64

dtypes: float64(3), int64(5)

memory usage: 81.9 KB

Survived: 2 valeurs uniques

Pclass: 3 valeurs uniques

Sex: 2 valeurs uniques

Age: 98 valeurs uniques

sibsp: 7 valeurs uniques

Parch: 8 valeurs uniques

Fare: 281 valeurs uniques

Embarked: 3 valeurs uniques

	Nombre	Pourcentage
Survived	0	0.00
Pclass	0	0.00
Sex	0	0.00
Age	0	0.00
sibsp	0	0.00
Parch	0	0.00
Fare	0	0.00
Embarked	2	0.15

Passagers avec Embarked manquant :

	Survived	Pclass	Sex	Age	sibsp	Parch	Fare	Embarked
61	1	1	1	38.0	0	0	80.0	NaN
829	1	1	1	62.0	0	0	80.0	NaN

Imputé avec le mode : 2.0

NaN restants : 0

Doublons exacts : 209

Passagers avec Fare = 0 : 17

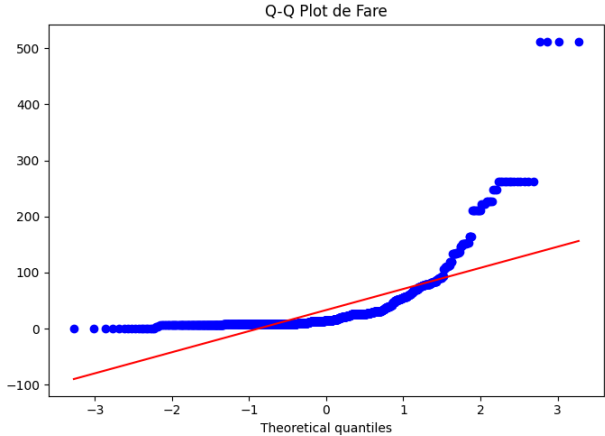
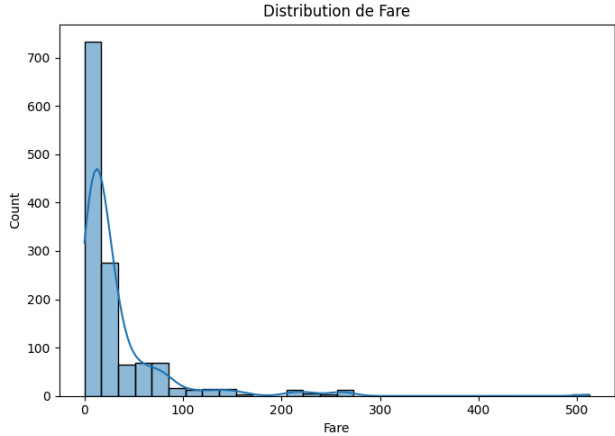
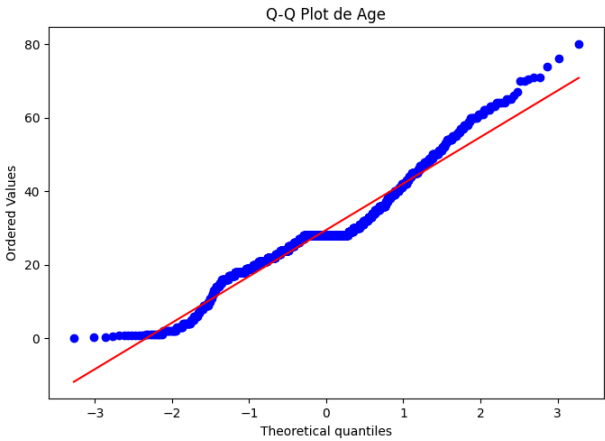
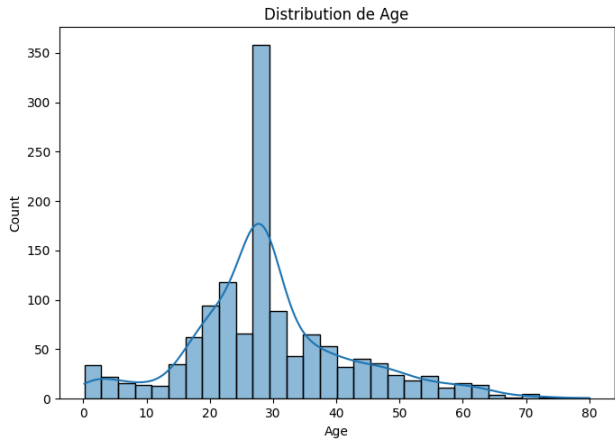
	Pclass	Age	Sex
179	3	36.0	0
263	1	40.0	0
271	3	25.0	0
277	2	28.0	0
302	3	19.0	0
413	2	28.0	0
466	2	28.0	0
481	2	28.0	0
597	3	49.0	0
633	1	28.0	0
674	2	28.0	0
732	2	28.0	0
806	1	39.0	0
815	1	28.0	0
822	1	38.0	0
1157	1	28.0	0
1263	1	49.0	0

Age minimum : 0.17

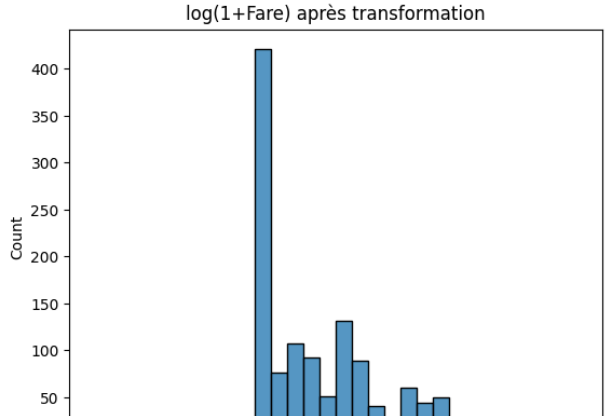
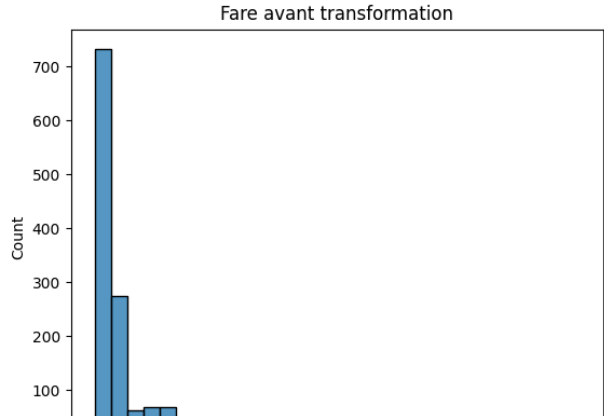
	Age	Fare
count	1309.00	1309.00
mean	29.50	33.28
std	12.91	51.74
min	0.17	0.00
25%	22.00	7.90
50%	28.00	14.45
75%	35.00	31.28
max	80.00	512.33

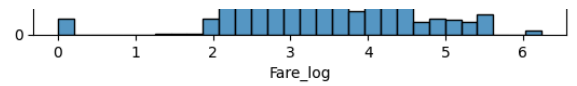
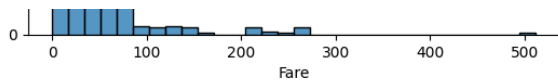
Age - Moyenne: 29.50, Médiane: 28.00, Mode: 28.0
 Ecart : 1.50
 Std pop: 12.90 | Std échantillon: 12.91
 Q1: 7.90, Q3: 31.27, IQR: 23.38
 Bornes Tukey: [-27.17, 66.34]
 Nb outliers: 171
 Billet le plus cher: 512.33
 Age - Skew: 0.54, Kurtosis: 0.97
 Fare - Skew: 4.37, Kurtosis: 27.05
 Outliers 3-sigma: 38

	Min	Q1	Médiane	Moyenne	Q3	Max	Std	Skewness	IQR
Age	0.17	22.0000	28.0000	29.503186	35.0000	80.0000	12.905241	0.540987	13.0000
Fare	0.00	7.8958	14.4542	33.281086	31.275	512.3292	51.741500	4.369510	23.3792

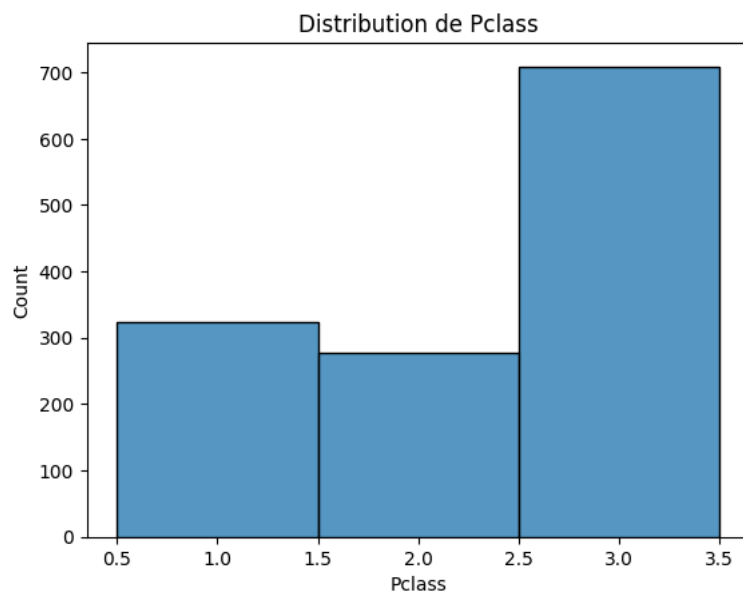
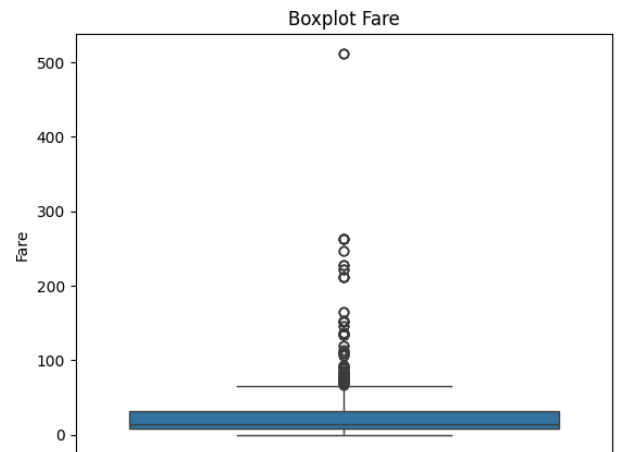
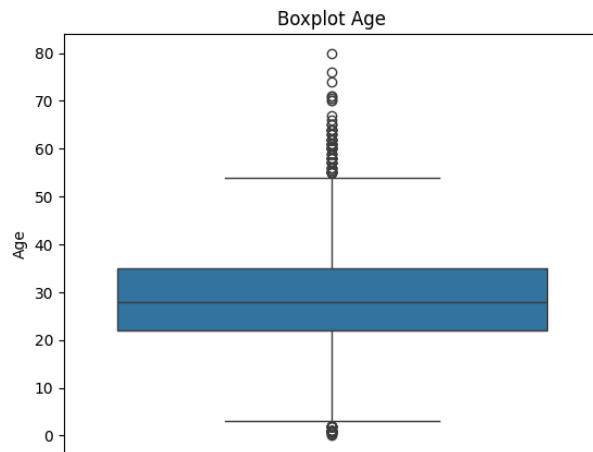


Shapiro-Wilk Age : stat=0.9553, p-value=0.0000
 Skewness avant: 4.37 | après: 0.54





Shapiro après log: p-value=0.0000

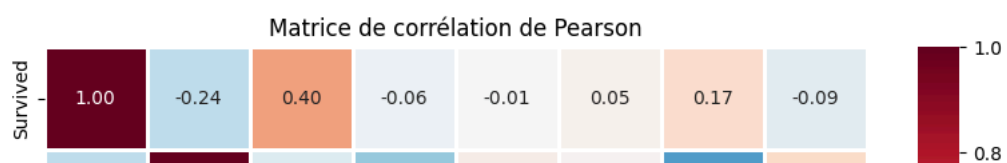
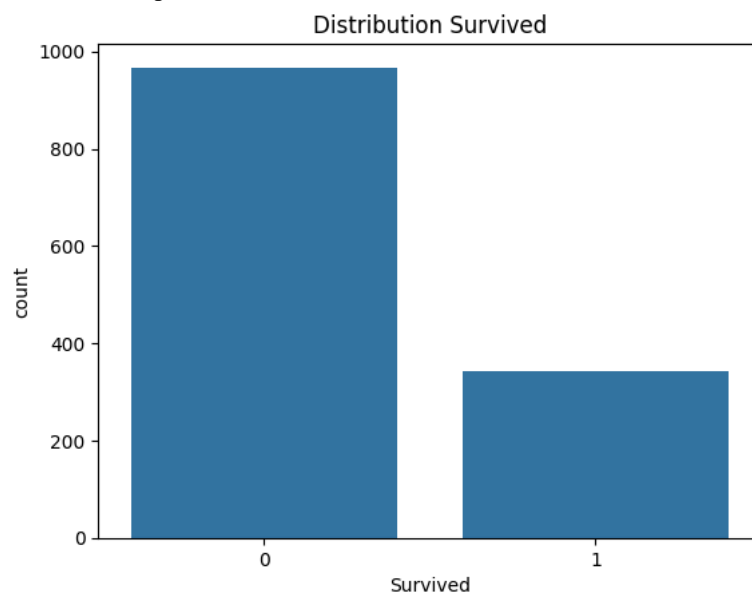


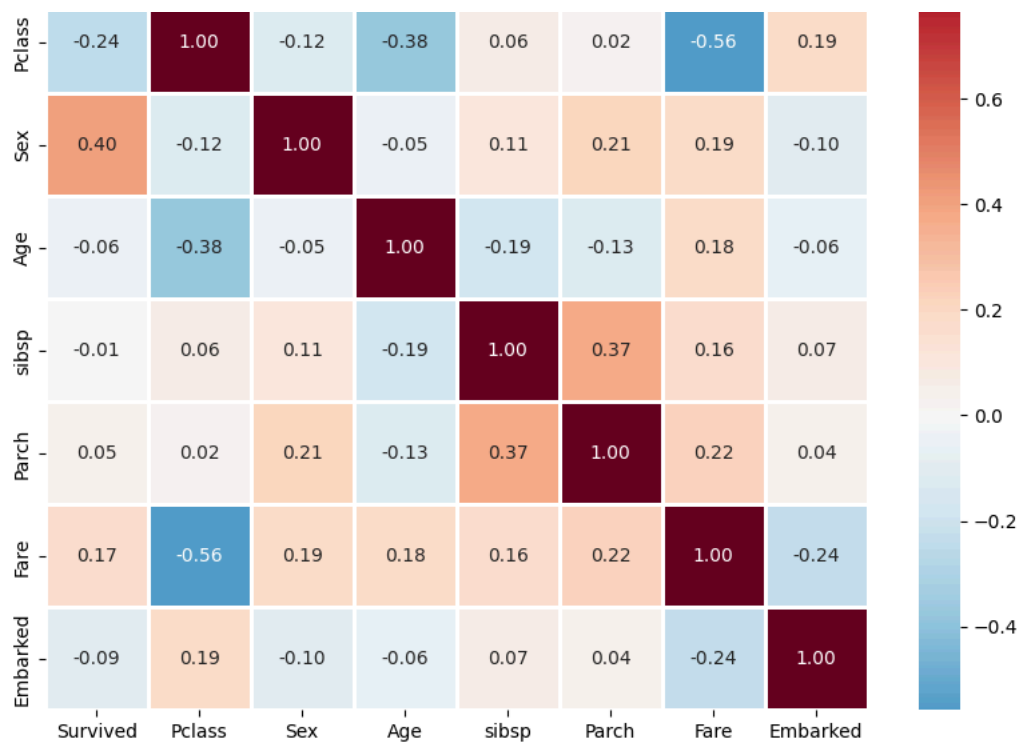
Pclass

1 24.7
2 21.2
3 54.2

Name: proportion, dtype: float64

Taux de survie global : 26.1%





Corrélations avec Survived:

Survived 1.000000

Sex 0.404020

Fare 0.173786

Parch 0.054908

sibsp -0.014375

Age -0.055862

Embarked -0.091394

Pclass -0.244686

Name: Survived, dtype: float64

Pearson Fare-Survived: 0.174

Spearman Fare-Survived: 0.242

Corr family_size: 0.021

Corr sibsp: -0.014 | Corr Parch: 0.055

Nombre de passagers seuls: 790

Chi2 Pclass-Survived: 78.79, p-value: 0.0000, ddl: 2

Chi2 Sex-Survived: 211.75, p-value: 0.0000

Shapiro survivants p=0.0000, morts p=0.0000

Levene variances égales p=0.0203

t-test Age: t=-1.937, p=0.0533

t-test Fare: p=0.0000

Mann-Whitney U: p=0.0000

ANOVA Fare~Pclass: F=373.13, p=0.0000

Multiple Comparison of Means - Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
1	2	-66.3298	0.0	-74.2669	-58.3927	True
1	3	-74.2045	0.0	-80.7109	-67.6981	True
2	3	-7.8747	0.0198	-14.7422	-1.0072	True

Chi2 Port-Class: 204.48, p=0.0000

age_group

Enfant 42.9

Adolescent 34.9

Jeune adulte 23.1

Adulte 29.1

Senior 18.2

Name: Survived, dtype: float64

Corr Fare: 0.174 | Corr log(Fare): 0.238

% passagers seuls: 60.4%

Taux survie seuls: 20.6%

Corr fare_per_person: 0.157 vs Fare: 0.174

/tmp/ipykernel_4309/787468009.py:297: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Please use observed=True to suppress this warning.

survival_by_agegroup = df.groupby('age_group')['Survived'].mean() * 100

Qui a survécu au naufrage du Titanic?

